Final Project

AI-Driven Intrusion Detection
Threat-Specific Detection Models on Real Telemetry

Dr. David Movshovitz & Efi Pecani | June 2026

Course 3917 – Using AI for Malware and Intrusion Detection
Reichman University, Efi Arazi School of Computer Science
Semester 2, 2026

Preliminary Document Due	June 14th, 2026
Final Submission Due	August 15th, 2026
Weight	55% of final course grade
Submission Format	Group_XX_Final_Project.zip (code repo + technical report + AI conversation logs)
Collaboration	Teams (group size per course policy)
AI Tools	Permitted with full conversation log disclosure (see AI Policy below)

Note - Use of Generative AI Tools

We encourage the use of GenAI tools as a learning aid. This includes:
- Standalone chat tools: ChatGPT, Claude, Gemini, Perplexity, etc.
- Agentic / IDE-integrated tools: Claude Code, Cursor, GitHub Copilot, Windsurf, etc.
- Local LLMs run via Ollama, LM Studio, Hugging Face, etc.

However, if you use any such tool you must submit alongside your project a separate conversation log file (.txt or .json) capturing the FULL conversation(s) you had with the AI tool while solving and debugging this project. This is non-negotiable - we want to understand HOW you used agentic /

LLM-driven development, not just whether the final code works.

If you used multiple tools, submit one log per tool (e.g., chatgpt_log.txt, claude_code_log.json). For agentic tools that don't expose a single export, dump session transcripts or screenshots of key conversations. Truncated, summarized, or curated logs will be treated as missing.

Project Overview

For your final project in this course, you are tasked with designing and implementing an AI-driven detection model tailored to a specific threat profile. Your project deliverables must demonstrate how specific machine learning and deep learning models and techniques can be applied to log data to effectively classify and detect a cyber attack.

MITRE ATT&CK Alignment (Highly Recommended)

We highly recommend aligning your project with the MITRE ATT&CK framework. However, please be aware that not all MITRE techniques and sub-techniques are visibly reflected or detectable within publicly available log telemetry. Choose a technique whose behavioral artifacts you can actually observe in the data you have access to.

Project Preparation: Threat Profile and Dataset Selection

Your first objective is to select a primary attack category and its associated techniques that can be effectively detected using either network or host logs. You must ensure that there are viable public datasets available for your chosen target threat.

In parallel to exploring your target attack techniques, you need to identify relevant, publicly available log data such as system event logs, network traffic metadata, or application-level traces that contains the necessary security telemetry to detect the malicious behaviour.

Deliverables for June 14th, 2026

You must submit a preliminary document containing the information listed below.

1. Group Members: List all participating students in your team.
2. Chosen Attack Technique: Identify the specific attack and its corresponding MITRE ATT&CK ID (if applicable).
3. Attack Characteristics & Log Mapping: Define the main characteristics of the attack and explicitly explain how these behaviours are reflected in network or host logs.
4. Target Datasets: Identify two publicly available datasets that you will use for training and testing your ML/DL models.
5. Academic Literature Validation: Provide citations for two peer-reviewed academic papers that have successfully implemented ML and/or DL techniques to detect this specific attack technique.

> *We will review these submissions together in class to ensure your project scope is on the right track. Use this checkpoint to validate your dataset access, paper availability, and feasibility of detecting the attack in the chosen telemetry before committing to the full implementation.*

Project Steps

In Steps 1 to 4, you will step into the role of a security data scientist to prepare your data for robust model training. Machine learning and deep learning models for intrusion detection are only as good as the features fed into them. Therefore, your task is to perform thorough exploratory data analysis (EDA), feature selection, and feature engineering.

Step 1: Threat Characterization and Telemetry Mapping

Your selected features must directly reflect the underlying mechanics and the characteristics of the cyber attack you want to detect.

Deep-Dive Attack Analysis: Students must move beyond high-level, surface-level definitions of their chosen threat. You are required to conduct an in-depth analysis of the attack's underlying mechanisms to determine exactly which telemetry sources are required for visibility, and precisely how the adversarial behavior manifests within those logs.

Feature and Detection Mapping: Based on the specific attack techniques selected, isolate the unique technical characteristics, artifacts, or anomalies left behind by the adversary. You must create a rigorous mapping between each attack characteristic and its exact representation in the data. This includes identifying specific raw packet fields (PCAP), NetFlow attributes, host-based log events (e.g., Sysmon, Windows Event Logs), or engineered features derived from the telemetry.

Concrete Examples for Guidance

Network-Based: C2 over DNS

If targeting DNS-based Command and Control, an adversarial characteristic like "Data exfiltration / beaconing via encoded subdomains" maps to an engineered ML feature such as the Shannon entropy score of the DNS query string or mean query length over a 5-minute window.

Network-Based: Port Scanning

If targeting network reconnaissance, an attack characteristic like "Rapid, automated connection attempts to multiple distinct ports" maps to NetFlow attributes like count of unique destination ports per source IP within a time window.

Host-Based: Credential Dumping (LSASS Minidump)

If targeting credential theft, the attack characteristic "Unauthorized memory dumping of the LSASS process" maps to a host log event like Sysmon Event ID 10 (ProcessAccess) where the TargetImage is lsass.exe and the GrantedAccess mask indicates generic read permissions (0x1410 or 0x1010).

Host-Based: Ransomware (Mass File Encryption)

If targeting ransomware execution, the attack characteristic "Rapid modification and renaming of user documents" maps to a high frequency of Sysmon Event ID 11 (FileCreate) or Event ID 26

(FileDelete/Rename) within a localized time cluster, or an engineered feature measuring the variance of file extension changes per minute.

Section Deliverable

Students must submit a Telemetry & Feature Mapping List (formatted as a structured table). Each item in the list should include the following:

1. Adversarial Behavioral Characteristic: A technical description of what the attack actually does at the OS or network level.
2. Required Telemetry Source: (e.g., Zeek DNS log, Cisco NetFlow v9, Windows Sysmon, etc.).
3. Specific Log Attributes / Raw Fields: The exact fields in the log where this is visible (e.g., query, GrantedAccess, dst_port) or that should be used for deriving / engineering a relevant feature (as applicable).
4. Derived / Engineered Feature: The actual mathematical or structural feature(s) you will generate for your ML models.
5. Detailed Explanation of the relationship between the adversarial characteristic and the suggested feature(s).

> *Note: there may be several features related to a single adversarial behavioural characteristic.*

Step 2: Academic Literature Review and Feature Extraction

As part of your project preparation, you were required to select two peer-reviewed academic papers relevant to your chosen threat category. In this step, you will bridge academic theory with practical engineering.

Feature Extraction Matrix: Review both papers deeply and extract the exact features the authors utilized to train their machine learning or deep learning models.

Comparative Feature Analysis: Document these features in a structured format and analyze the rationale behind the authors' choices. You must answer:

1. What semantic security meaning does each feature hold according to the authors?
2. How do the features from Paper A compare or contrast with the features in Paper B?
3. Are there features proposed by the authors that you can directly adopt, modify, or expand upon for your own datasets?

Step 3: Exploratory Data Analysis (EDA) and Feature Justification

Before training any model, you must gain data intuition and empirically justify your feature choices using your project's datasets.

Exploratory Data Analysis (EDA): Perform a comprehensive statistical and visual analysis of your dataset. This must include plotting distributions, calculating class variances, and mapping out correlations between your proposed features and the target labels (Benign vs. Malicious). As

part of your EDA, you must systematically identify redundant or noisy features, engineer new contextual indicators (as required), and optimize the feature list you have generated in Steps 1 and 2 (to maximize detection accuracy while minimizing computational overhead).

Note: please be aware to include features that are related to the specific testbed used to generate the log datasets like specific IP addresses, specific Process IDs, specific timing info, etc.

Empirical Feature Justification / Optimization: For every feature you suggested in Step 1 (and those adopted from Step 2), you must provide a rigorous, data-driven justification based on your EDA.

> **Hard Empirical Justification Required**
>
> You must prove that the feature shows a measurable, statistically significant difference in behaviour between benign traffic / usage and the malicious attack traffic / usage. Broad, hand-wavy theoretical justifications without data-driven evidence (e.g., charts, statistical summaries) will not be accepted.

Step 4: Machine Learning-Based Feature Ranking and Verification

To validate your domain-driven engineering choices, you will let a machine learning algorithm evaluate the utility of your features.

Algorithmic Feature Ranking: Apply a tree-based machine learning technique that inherently generates feature importance scores (e.g., Random Forest Gini Importance / Mean Decrease in Impurity, or XGBoost Feature Importance) to your datasets.

Comparative Evaluation and Analysis: Create a visualization (such as a horizontal bar chart) ranking all your features based on the model's evaluation. Compare this algorithmic ranking against the domain-expert features you manually selected and justified in Steps 1, 2, and 3.

Critical Analysis Report: Write a detailed analysis addressing the following:

1. Did the algorithm align with your security intuition? (i.e., did the features you expected to be highly reflective of the attack actually rank at the top?)
2. Identify any "surprises" - features the algorithm ranked incredibly high that you did not expect, or features you thought were critical but the algorithm ignored.
3. Explain why these discrepancies occurred. Is it due to a data leakage issue (e.g., a specific IP address or timestamp artifact that the model is exploiting)? Is it due to multicollinearity (two features carrying identical information)? Or did the model uncover a latent behavioural pattern you missed?

Step 5: Multi-Dataset Sourcing & Harmonization

The "Common Feature" Challenge: Because different cybersecurity datasets utilize entirely different schema headers, logging formats, and collection granularities, your team must establish

a Unified Feature Schema. You are required to implement a robust preprocessing script that extracts, derives, or calculates an identical set of features (e.g., flow duration, packet inter-arrival time, payload entropy) from both of your selected datasets. This ensures your downstream models remain invariant to the underlying data source.

Mapping Requirement: Explicitly map this unified feature schema back to the feature engineering and literature review findings completed in Steps 1 to 4.

Cross-Dataset Distribution Analysis: For every feature defined in your unified schema, conduct a comparative statistical and visual exploration across both datasets.

1. Examine how the feature's characteristics, mean, variance, and distribution shapes differ between the two datasets (e.g., due to different network topologies or background noise).
2. Document these differences and prescribe an explicit engineering remedy (e.g., Z-score normalization, MinMax scaling, log transforms, or other robust scaling) to mitigate dataset-specific bias before feeding the data into your models.

Step 6: Model Selection and Academic Justification

Your team must select, implement, and evaluate four (4) distinct ML / DL models, ensuring your selection includes at least one architecture from each of the following paradigms:

1. Traditional Supervised Learning: e.g., Random Forest, XGBoost.
2. Deep Learning Architectures: e.g., RNN / LSTM for sequential analysis, 1D-CNN for spatial analysis.
3. Unsupervised / Anomaly Detection: e.g., Isolation Forest or Deep Autoencoders.

Section Deliverables

Structural & Cyber-Threat Justification: Your choice of models must not be a blind, brute-force experiment. You are required to explicitly justify why a specific architecture's mathematical and structural properties make it uniquely suited for the attack vectors you are targeting. For each of the four selected models, explain how its inherent advantages (e.g., the temporal dependency tracking of LSTMs for beaconing / low-and-slow attacks, the spatial pattern recognition of CNNs, or the non-linear high-dimensional boundary mapping of ensemble methods) align with both the threat characteristics and the structure of your features.

Academic Literature Grounding: To anchor your practical implementation in current cybersecurity research, perform a literature check within peer-reviewed academic repositories (e.g., IEEE Xplore, ACM Digital Library, Google Scholar). For each of your four selected models, locate at least one relevant academic study that applied that exact architecture to your chosen cyber threat. In your final report, synthesize these findings: detail how those researchers configured their model architectures, the datasets they utilized, and the specific performance metrics (e.g., Detection Rate / Recall, False Positive Rate, F1-Score) they achieved.

Step 7: Pipeline Implementation

Modular Pipeline Architecture: Design and implement a highly modular, clean, and reproducible code pipeline following this strict sequence:

```
Data Ingestion -> Preprocessing & Normalization -> Feature Selection &
Engineering -> Model Training -> Evaluation
```

Dataset Dependency Rule

While the Data Ingestion module will naturally contain dataset-specific parsing logic, all subsequent modules (Preprocessing, Selection, Training, Evaluation) must be entirely agnostic of the dataset provided as input. A grader should be able to swap one dataset for another and have the rest of the pipeline run unchanged.

Algorithmic Requirements

Class Imbalance Mitigation: Cyber attacks are fundamentally "needles in a haystack." Your pipeline must programmatically handle severe class imbalance during training (e.g., via cost-sensitive learning, class weighting, or sampling techniques like SMOTE).

No Data Leakage

To avoid data leakage, any sampling or scaling must be fit strictly on the training fold and applied to the testing fold. Wrap this in an sklearn Pipeline so that cross-validation splits do not leak test distribution into your scaler or sampler fits.

Explicit Hyperparameter Configuration: You must explicitly define and configure the hyperparameters for each of your four models. Do not rely on the default library parameters (e.g., default scikit-learn or XGBoost settings).

Sensitivity Analysis: Systematically test and document how altering key hyperparameters (e.g., tree depth, learning rates, regularization parameters, or network depth) affects model detection accuracy and stability.

Documentation and Reproducibility Deliverable

Your submission must include a detailed technical implementation chapter covering:

1. A visual architectural diagram and text explanation of your modular pipeline.
2. The specific development frameworks, packages, and hardware / software environments utilized.
3. Your exact methodology for mitigating class imbalance, along with a theoretical defense of why you chose that method.
4. The validation strategy (e.g., stratified k-fold cross-validation) and metrics (Precision, Recall, F1, ROC-AUC, FPR) chosen for testing model robustness.

5. A comprehensive breakdown of the chosen hyperparameters, the rationale behind their selection, and an analysis of their empirical impact on model performance.

Step 8: Comparative Performance, Error Analysis, and Intelligent Ensemble Optimization

The final stage of this project requires your team to critically evaluate your models, diagnose their systematic failures, test their generalizability across distinct environments, and engineer a high-fidelity, production-ready hybrid ensemble pipeline.

Deep Error Analysis & Pipeline Refinement

You must conduct a forensic, sample-level error analysis for each of your four models across both datasets. Move beyond aggregate metrics to understand exactly why your models fail.

Error Categorization: Break down your confusion matrices. Are specific attack sub-categories (e.g., a low-and-slow C2 beacon vs. an aggressive automated scan) consistently causing False Negatives (FN)? Is a specific type of benign background traffic (e.g., automated administrative scripting or heavy database queries) causing a high False Positive Rate (FPR)?

Explanatory Diagnostic Research: Extract and inspect the raw log data / feature values for the specific samples where the models failed. Document the shared characteristics of these misclassified logs. You must analyze how and why these failure modes vary between different model architectures (e.g., why a Random Forest failed on a sample that an LSTM successfully classified).

Iterative Optimization: Based on your diagnostic findings, execute a targeted refinement of your pipeline. Document how you adjusted your feature engineering, data normalization, or class-imbalance mitigation to suppress these systematic errors. Implement these enhancements and provide "before-and-after" metrics to prove optimization.

Cross-Dataset & Literature Benchmarking

Cross-Dataset Variance Analysis: Evaluate and compare the performance metrics (Accuracy, Precision, Recall / Detection Rate, and FPR) of your four implemented models across both selected datasets. If a model achieves a 99% F1-score on Dataset A but plummets to 60% on Dataset B, you must diagnose the root cause. Detail whether this performance degradation is driven by Environmental Distribution Shift (e.g., differing network topologies, background noise levels, or structural variations in the attack execution) or systemic Model Overfitting to the primary training set.

Literature Benchmarking: Benchmark your empirical results against the academic findings you extracted during your Step 2 literature review. Critically discuss any discrepancies between your metrics and the authors' reported benchmarks, attributing variances to differences in data normalization, feature selection, hyperparameter choices, etc.

Advanced Ensemble Strategy & Local LLM Integration

The capstone engineering task is to fuse your independent models into a unified, resilient security architecture designed to maximize detection fidelity while minimizing analyst fatigue (false positives). Your architecture must programmatically incorporate the following two paradigms:

Hybrid Behavioural Cascading: Design and implement a multi-stage logical pipeline where your models complement one another's operational strengths.

Example Cascade Architecture

A fast, lightweight Unsupervised Model (e.g., Isolation Forest) acts as a high-sensitivity, first-line filter to instantly flag statistical anomalies at scale. These flagged anomalies are then immediately routed to your Supervised Model (e.g., XGBoost) or Deep Learning Model to accurately categorize the specific threat type or confidently discard benign, ambient noise.

LLM-Driven Triage and Contextual Arbitration: Integrate an open-source Large Language Model into your ensemble as an intelligent triage layer.

1. The LLM must serve as a contextual arbitrator when your primary models experience high-uncertainty edge cases or direct disagreements (e.g., when the Supervised model predicts "Benign" but the Deep Learning model flags "Malicious").
2. Your pipeline must format and pass the anomalous feature context, payload metadata, or associated security log strings to the LLM. The LLM must be prompted to perform structured chain-of-thought reasoning over this security telemetry to deliver a final, explainable classification decision.

Recommended LLM Hosting (Hugging Face Inference)

Rather than running a heavy local LLM, we strongly recommend using a Hugging Face Inference endpoint with a free / academic HF token. This gives you access to the latest Llama models (e.g., Llama 3.1 8B Instruct, Llama 3.3 70B Instruct) without burning your laptop or your battery.

Suggested setup:

1. Create a free account at huggingface.co and generate a personal access token.
2. Use the huggingface_hub Python client (InferenceClient) or the OpenAI-compatible HF Router endpoint.
3. Llama 3.1 8B Instruct via Inference Providers is fast and sufficient for arbitration.
4. Document your model choice and HF token usage in Chapter 8.5 (do NOT commit your token to the submitted code - read it from an environment variable or a .env file).

Latency Warning - Local LLMs

If you choose to run the LLM locally (Ollama, LM Studio, llama.cpp), be aware:
- Llama-3 8B at int4 quantization needs ~6 GB RAM / VRAM. Mistral-7B is similar.
- On CPU-only laptops, expect 5-30 seconds per inference - completely unworkable for any ensemble that calls the LLM more than a handful of times.
- On Apple Silicon or a discrete GPU, expect 1-3 seconds per inference - still slow.
- Your evaluation pipeline must NOT call the LLM on every test sample. Cascade properly: primary models classify confidently >> only ambiguous edge cases reach the LLM.
- Document the exact LLM call count per test run and the resulting wall-clock latency. If your latency makes the system production-impractical, you must say so and discuss the tradeoff.

Hugging Face Inference (above) avoids all of this and is our recommended path.

Section Deliverables

1. Error Analysis Report: A comprehensive section detailing the specific failure modes of each model, accompanied by a technical defense of the pipeline refinements you implemented to fix them.
2. Cross-Dataset Performance Matrices: Comparative tables and visualizations illustrating model performance across both datasets, alongside a detailed analysis of environmental drift and model generalizability.
3. Hybrid Pipeline Architecture Diagram & Code: A complete software block diagram illustrating your cascading and LLM-arbitration logic, alongside the fully implemented Python codebase.
4. LLM Configuration & Prompt Design Documentation: An explicit explanation of your chosen open-source LLM, its deployment constraints, the exact prompt engineering templates used for triage, and an evaluation of its reasoning accuracy on edge cases.

Final Project Submission Overview

Students must submit their final project as a single zipped archive containing three primary components: (where X1 and X2 are your IDs)

```
Group_X1_X2_Final_Project.zip
```

1. The Code Implementation Repository

1. A modular, fully commented Python codebase adhering to the strict Dataset Dependency Rule.
2. An execution entry point script (e.g., main.py or a sequence of clearly numbered Jupyter Notebooks).
3. A requirements.txt file specifying all external frameworks, packages, and exact library versions used.
4. A README.md file with quick-start execution setup instructions.

2. The Final Technical Project Report

A comprehensive Word Document (.docx) comprehensively mapping out the team's theoretical engineering, empirical findings, and architectural designs.

3. AI Conversation Logs

If you used any GenAI tool while working on this project (chat tools, agentic IDE tools, or local LLMs - see AI Policy), you must include the FULL conversation logs as separate files inside your ZIP submission.

1. File format: .txt or .json. One file per tool used.
2. Naming: ai_logs/<tool_name>_log.txt (e.g., ai_logs/chatgpt_log.txt, ai_logs/claude_code_log.json, ai_logs/cursor_log.txt).
3. Content: the full, unedited conversation. Include your prompts and the AI's responses. Do NOT summarize or curate.
4. Coverage: logs should span the project lifecycle - planning, debugging, error analysis, report writing, etc.
5. If you used the same tool across many sessions, concatenate the sessions in chronological order with a clear session separator.

> *Why we require this: we want to estimate how you actually used AI / agentic development on the project, not just whether the final code works. Submissions with no logs but obvious AI fingerprints in the code or report will be treated as undisclosed AI use and penalized. Submissions with rich logs that show real iterative problem-solving will receive credit for that demonstrated process.*

Final Technical Report Template

Formatting Instructions

1. Technical report is limited to 15 pages in 1.5 spacing.
2. Use a clean, professional typography setup (e.g., Arial or Calibri with font size 11/12).
3. Use bold headings, clear section breaks, and properly labelled code snippets, equations, or mathematical notation where appropriate.
4. Use short and clear answers.
5. Use tables as much as possible.
6. You can provide the supporting exploration graphs in appendixes as required.
7. All tables and figures must have a descriptive caption.

> *Note: Report quality and adherence to formatting are graded - see the Title Page item in the rubric below.*

Required Report Structure

Title Page

Course Name: Using AI for Intrusion and Malware Detection
Project Title: [Insert Selected Attack / Threat Profile and MITRE ATT&CK ID] (e.g., AI-Driven Detection of C2 Communication over DNS - T1071.004)
Group Identification: Group Number [XX]
Team Members: [Student Name 1 & Email, Student Name 2 & Email, etc.]
Date of Submission: [Insert Date]

Executive Summary (5 points)

(Maximum: 1 page) Provide a concise executive and technical overview of the project. State the targeted cyber threat, the two public datasets utilized, the four core machine learning / deep learning models deployed and their results, and a high-level summary of the final hybrid ensemble pipeline's optimization performance, and its results.

Chapter 1: Threat Characterization and Telemetry Mapping (Step 1) (15 points)

1.1 Deep-Dive Threat Analysis: Define your chosen attack profile and sub-techniques using the MITRE ATT&CK framework. Explain the lower-level operational mechanics of how the attack executes at the operating system or network layer.
1.2 Structural Telemetry Mapping List / Table: Present your structured Telemetry & Feature Mapping List exactly as prescribed in the project description.
1.3 Theoretical Feature Rationale: For each engineered feature in your list/matrix, write a detailed technical narrative explaining the precise semantic security relationship between the malicious behavior and the mathematical or structural representation of the feature.

Chapter 2: Academic Literature Review & Feature Extraction (Step 2) (10 points)

2.1 Literature Extraction Matrix: Create a comparative matrix documenting the exact features and models extracted from your two selected peer-reviewed academic papers.

2.2 Comparative Analytical Essay: Answer the following three analytical questions deeply:

- What semantic security meaning do the authors assign to their chosen features?
- Contrast the engineering approaches of Paper A vs. Paper B.
- Detail which features you adopted, modified, or rejected for your own practical implementation, defending your reasoning.

Chapter 3: Exploratory Data Analysis (EDA) & Domain Justification (Step 3) (15 points)

3.1 Statistical and Visual Exploration: Document your visual and data profiling efforts across both datasets. Include annotated charts showing class distributions, feature variance plots, and a correlation matrix heatmap.

3.2 Noise & Redundancy Reduction Analysis: Document your systematic process for filtering out redundant or uninformative telemetry, and show how you optimized your feature list to minimize downstream computing costs.

3.3 Empirical Justification Report: For every feature proposed, provide hard empirical evidence (e.g., box plots, class-conditional histograms, or statistical tests) proving that the feature exhibits a measurable, statistically significant difference in behavior between benign usage and malicious traffic (or between the malicious variants).

Chapter 4: Algorithmic Feature Ranking and Verification (Step 4) (10 points)

4.1 Tree-Based Feature Importance Ranking: Embed your horizontal bar chart visualizing the feature importance scores generated by an ensemble, tree-based machine learning algorithm (e.g., Random Forest MDI or XGBoost importance).

4.2 Critical Discrepancy Analysis Report: Compare this algorithmic ranking to your manual, domain-expert selection from Chapters 1 to 3. Write a comprehensive report covering:

- Alignment with your security intuition.
- Detailed diagnostic investigation of any "surprises" (features over-ranked or under-ranked by the model).
- Technical verification regarding whether anomalies are driven by data leakage (e.g., specific IP artifacts, MAC addresses, or timestamp bounds), multicollinearity, or genuine latent behavioral patterns.

Chapter 5: Multi-Dataset Sourcing & Harmonization (Step 5) (10 points)

5.1 Unified Feature Schema Definition: Define the engineering specifications of your Unified Feature Schema used to resolve the header and logging differences across your two source datasets. Map this final unified schema back to the findings of your previous phases.

5.2 Cross-Dataset Distribution Shift Analysis: Present side-by-side distribution plots comparing how the same unified feature behaves across Dataset A vs. Dataset B.

5.3 Data Scaling & Scaling Remedies: Document the mathematical normalization techniques (e.g., Z-score, MinMax, or other robust transforms) implemented to suppress environment-specific biases and prevent down-stream model skew.

Chapter 6: Model Selection and Academic Grounding (Step 6) (5 points)

6.1 Architectural Customization Justification: For each of your four distinct models (Traditional Supervised, Deep Learning, and Unsupervised / Anomaly Detection), provide a mathematical and structural explanation of why that specific algorithm aligns natively with your threat characteristics.

6.2 State-of-the-Art Literature Synthesis: Present your literature check synthesizing how academic researchers configured these exact architectures for your chosen threat profile. Summarize their configurations, datasets, and benchmark evaluation metrics.

Chapter 7: Modular Pipeline Implementation (Step 7) (10 points)

7.1 Software Pipeline Block Diagram: Provide a visual software architecture diagram detailing your modular, zero-dependency data pipeline sequence: Data Ingestion -> Preprocessing & Normalization -> Feature Selection & Engineering -> Model Training -> Evaluation. Prove textually how your code maintains absolute abstraction from the input dataset once out of the ingestion layer.

7.2 Class Imbalance Strategy and Validation Framework:

- Document the algorithmic mitigation approach used to counter the "needle in a haystack" problem (e.g., cost-sensitive weights, SMOTE).
- Detail how you structurally isolated your scaling and sampling parameters entirely to the training folds to ensure zero data leakage into your cross-validation folds.

7.3 Hyperparameter Sensitivity Report: Enumerate your explicit, custom hyperparameter configurations (avoiding default package settings). Document your sensitivity experiments, analyzing how variations in model configurations directly shifted accuracy and system stability metrics.

Chapter 8: Performance Evaluation, Error Analysis, and Hybrid Ensemble (Step 8) (20 points)

8.1 Forensic Error Categorization (Confusion Matrices): Display full confusion matrices for all models across both datasets. Isolate and explain systemic False Negatives (FN) across attack sub-categories and False Positives (FP) triggered by specific administrative background traffic. Document the sample-level diagnostic investigation and the subsequent pipeline optimizations you verified to fix them.

8.2 Cross-Dataset Variance Analysis: Provide a primary, comprehensive evaluation comparison table covering Accuracy, Precision, Recall / DR, and FPR across both datasets. Diagnose any performance drops, explicitly defining if variances are due to Environmental Distribution Shift or systemic Model Overfitting.

8.3 Literature Benchmarking Discussion: Directly contrast your empirical results against the academic literature benchmarks reviewed in Chapter 6, defending any structural discrepancies.

8.4 Hybrid Behavioral Cascading Architecture: Provide the design rationale, block layout, and code implementation explanation for your multi-stage cascading logic pipeline.

Bonus Chapter: LLM-Driven Triage & Contextual Arbitration (+10 points)

This chapter is OPTIONAL. Teams that successfully integrate an LLM arbitration layer can earn up to +10 bonus points on top of the 100-point base.

B.1 LLM Hosting & Configuration: Document your chosen LLM, hosting approach (Hugging Face Inference recommended; local Ollama / LM Studio acceptable with latency analysis), model family and size, and deployment constraints. Do NOT commit any HF tokens to the codebase.
B.2 Prompt Engineering: Provide the explicit prompt templates used to pass anomalous context and drive structured, chain-of-thought reasoning over model disagreements.
B.3 Reasoning Accuracy: Critically analyze the LLM's classification accuracy on high-uncertainty edge cases. Compare LLM verdicts against your supervised / unsupervised models and against ground truth.
B.4 Operational Tradeoff: Report wall-clock latency per LLM call, total LLM call count per evaluation run, and discuss whether this architecture would be viable in a real production SOC. Honesty about limitations is rewarded; hand-waving is not.

Appendix

Appendix A: Code Execution Instructions: Concrete, step-by-step commands to spin up the local environment, initialize the data pipelines, and invoke the LLM framework.
Appendix B: Frameworks and Hardware Environment Specs: Specification of all foundational libraries (e.g., scikit-learn, PyTorch, XGBoost, Ollama) and underlying compute execution hardware.

Grading Rubric Summary

Component	Points
Executive Summary	5
Chapter 1 - Threat Characterization & Telemetry Mapping	15
Chapter 2 - Academic Literature Review	10
Chapter 3 - EDA & Domain Justification	15
Chapter 4 - Algorithmic Feature Ranking	10
Chapter 5 - Multi-Dataset Sourcing & Harmonization	10
Chapter 6 - Model Selection & Academic Grounding	5
Chapter 7 - Modular Pipeline Implementation	10
Chapter 8 - Evaluation, Error Analysis & Hybrid Ensemble	20
Total (base)	100
Bonus Chapter - LLM-Driven Triage & Arbitration	+10
Total (with bonus)	110

AI Policy & Academic Integrity

Use of Generative AI Tools

We encourage the use of GenAI tools as a learning aid. This includes standalone chat tools (ChatGPT, Claude, Gemini), agentic / IDE-integrated tools (Claude Code, Cursor, GitHub Copilot, Windsurf), and local / hosted LLMs (Ollama, LM Studio, Hugging Face).

If you use any such tool, you must submit the FULL conversation logs alongside your project as separate .txt or .json files (one per tool, e.g., ai_logs/chatgpt_log.txt, ai_logs/claude_code_log.json). Full means unedited - we want to estimate how you actually used agentic / LLM-driven development. Truncated, summarized, or curated logs will be treated as missing.

Blind copy-pasting without understanding will be penalized. Submissions with no logs but obvious AI fingerprints in the code or report will be treated as undisclosed AI use.

1. Copying from other groups is not permitted.
2. You may consult: course materials, lecture slides, library documentation, Stack Overflow, and published peer-reviewed papers on your chosen attack technique.
3. Submissions that are clearly copy-pasted without understanding (AI-generated or otherwise) will receive a grade of zero.

Good luck. The best detection systems aren't the ones with the highest F1 score on a single benchmark - they're the ones whose authors can explain exactly when and why they fail.